TAI4SE Assignment 1 : Detecting near-duplicate Python files in The Stack v2

Saumit Paul
Department of Electrical and Computer Engineering
North Carolina State University
sspaul3@ncsu.edu

Abstract

This report studies duplicate source files as a data-quality problem in code corpora. I implemented a Python detector that represents each file using token 5-grams and compares file pairs using Jaccard similarity. The detector uses a threshold of 0.70, following the near-duplicate threshold reported for The Stack v2. On the provided 500-file starter set, it found all five known duplicate pairs without flagging any additional pairs. I then applied the detector to a reproducible 2,496-file Python sample drawn from 200 repositories in a public re-host of The Stack v2. The detector flagged 1,808 within-repository pairs and skipped 29 files that could not be processed. I manually inspected 15 high-scoring pairs and selected six representative pairs for submission.

1 Data-quality issue

The selected data-quality issue is source-code duplication. I treat a duplicate as a pair of files that contain substantially the same code, even when comments, formatting, or small parts of the code have changed. Exact file matching would therefore be too restrictive for this task.

Duplication reduces the amount of unique information in a code corpus. A dataset may contain a large number of files while repeatedly including the same or nearly identical code. This can distort statistics computed from the dataset and can also affect models trained or evaluated using it. Prior work has shown that duplicated code can make evaluations of machine-learning models on code overly optimistic [2].

The Stack v2 also performs near-duplicate detection as part of its data preparation process. It compares token 5-grams using Jaccard similarity with a threshold of 0.70 [1]. Because The Stack v2 is much larger, its pipeline first uses MinHash and locality-sensitive hashing to find files that are likely to be similar instead of comparing every possible pair. My experiment is small enough to calculate Jaccard similarity directly for every pair.

2 Detection method

Each Python file is first split into meaningful code tokens. Comments, whitespace, and formatting are ignored, so changing indentation or adding comments does not affect the comparison. Names and values in the code are left unchanged.

From the resulting token sequence, the detector creates overlapping groups of five tokens, called 5-grams. Each file is represented as the set of 5-grams that appear in it. For files A and B, similarity is calculated using the Jaccard index:

$$J(A, B) = \frac{|G_A \cap G_B|}{|G_A \cup G_B|},$$

where G_A and G_B are the 5-gram sets for the two files. A score of 1 means both files contain exactly the same set of 5-grams, while a lower score means they share less code. A pair is flagged when $J(A, B) \geq 0.70$. This threshold was chosen before evaluating the starter set because it matches the threshold reported for The Stack v2 [1].

For the corpus experiment, I used the public Python re-host at https://huggingface.co/datasets/Reset23/the-stack-v2-python, pinned to revision `ee79c0cd3580beba2d627a3a457ae265b0fc979b`. This is not the canonical BigCode release. I used only `data/train-00000-of-00011.parquet`.

I kept non-empty UTF-8 Python files no larger than 100,000 bytes. Repositories needed at least five eligible files. I then selected 200 repositories using a deterministic sampling procedure with seed 591. This avoids manually choosing repositories while ensuring that rerunning the sampling code selects the same repositories. At most 50 Python files were kept from each selected repository. Files were compared only with other files from the same repository.

3 Evaluation and findings

The provided starter set contains 500 Python files and five known duplicate pairs. The detector processed 498 files. Two files were rejected during tokenization and were therefore excluded from pairwise comparison. One of them, `0188.py`, contains an HTML webpage rather than Python code despite having a `.py` extension. The recorded run also rejected `0162.py`.

The detector compared all valid file pairs and found exactly the five labeled duplicate pairs.

File A	File B	Jaccard similarity
`0070.py`	`0339.py`	1.000
`0155.py`	`0322.py`	1.000
`0415.py`	`0456.py`	1.000
`0287.py`	`0393.py`	0.972
`0171.py`	`0278.py`	0.928

Table 1: Duplicate pairs found in the starter set.

Since all five known duplicate pairs were found and no extra pairs were flagged, precision, recall, and F1 are all 1.00 on this specific starter set. The starter set is small, so these scores should not be interpreted as the expected performance on larger code datasets.

The similarity scores also show a clear gap between the known duplicates and the remaining file pairs. The lowest-scoring duplicate had a similarity of 0.928, while the next highest pair scored only 0.355. This suggests that the 0.70 threshold separates the provided duplicate examples cleanly.

The pair `0171.py`/`0278.py` is a useful near-duplicate example. The two files contain the same overall program and control flow, but some function names, attributes, and values are changed. The detector still gives them a similarity score of 0.928. This shows that the method can detect files that are similar without being exact copies.

There were no false positives or false negatives among the files that the detector successfully processed. However, two files could not be processed at all. Any duplicate involving one of those files would therefore be missed.

The sampled Stack v2 slice contained 2,496 files from 200 repositories. The detector flagged 1,808 within-repository pairs at the 0.70 threshold and skipped 29 files that could not be tokenized.

I manually inspected the 15 highest-scoring pairs. Several of these were redundant because more than two files contained the same or nearly identical code, causing the same duplicate group to appear as multiple file pairs. I therefore selected six representative pairs for `candidates.jsonl`. Each pair contributes two file records, so the final file contains 12 records. The selected similarities range from 0.990 to 1.000.

Each record includes the source `blob_id`, encoding, repository name, file path, revision identifier, and the reason the file was flagged. I omit `line_span` because the detector treats duplication as a relationship between whole files rather than a problem on specific lines.

The 1,808 pairs are automatic detections, not 1,808 manually confirmed duplicates. I manually reviewed only 15 high-scoring pairs, so I do not estimate an overall accuracy for the Stack v2 sample from this manual review.

4 Limitations and implications

Limited scope. The corpus experiment uses only Python files from one shard of a public Stack v2 re-host. The reported counts therefore describe this sample and should not be treated as statistics for the full Stack v2 dataset.

Duplicates across different repositories. Files were compared only with other files from the same repository. This keeps the number of comparisons manageable, but it means the experiment will miss code that was copied between different repositories.

Heavily edited copies. The detector is not affected by changes to comments, whitespace, or formatting. However, changing many variable names, values, or sections of code lowers the Jaccard score. Two files with the same origin could therefore be edited enough to fall below the 0.70 threshold.

Despite these limits, the experiment shows that token-based similarity can identify both exact copies and lightly modified copies without requiring files to be identical. For much larger datasets, the same basic comparison would need a more efficient way to narrow down which file pairs should be checked.

References

[1] Anton Lozhkov et al. StarCoder 2 and The Stack v2: The Next Generation. *arXiv preprint arXiv:2402.19173*, 2024.

[2] Miltiadis Allamanis. The Adverse Effects of Code Duplication in Machine Learning Models of Code. In *Proceedings of the 2019 ACM SIGPLAN International Symposium on New Ideas, New Paradigms, and Reflections on Programming and Software*, pages 143–153, 2019.